

FULL STACK DEVELOPMENT

FASE 4 | AULA 04 -

SINGLE SPA - ROUTING

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMANIP

O QUE VEM POR AÍ?

Nesta aula, vamos mergulhar em um aspecto fundamental de qualquer aplicação frontend: o roteamento. Se você já trabalhou com frameworks como React ou Angular, sabe o quanto o roteamento é crucial para controlar a navegação e a experiência do usuário. No contexto do Single SPA e microfrontends, o roteamento ganha uma camada extra de importância e complexidade, e é exatamente isso que vamos explorar juntos.

Começaremos com uma introdução ao conceito de roteamento no Single SPA. Vou te mostrar como o Single SPA gerencia o roteamento em uma aplicação composta por múltiplos microfrontends, e porque isso é tão importante para garantir uma navegação fluida e coesa. Você aprenderá a configurar o roteamento básico, definindo rotas estáticas no root config, o que permitirá que sua aplicação carregue os microfrontends corretos com base na URL que o usuário está acessando.

Em seguida, avançaremos para a criação e configuração de rotas estáticas. Vamos ver como definir rotas que apontam para microfrontends específicos, garantindo que cada parte da aplicação seja carregada corretamente quando o usuário navegar para diferentes seções. Esse é um passo essencial para qualquer aplicação modular e distribuída como as que criamos com microfrontends.

Por fim, exploraremos o conceito de roteamento dinâmico no Single SPA. Vamos explicar o que torna o roteamento dinâmico tão poderoso, permitindo que a aplicação responda a mudanças de URL de forma mais flexível e adaptável. Você verá como configurar rotas dinâmicas e como utilizá-las para criar experiências de navegação mais ricas e interativas.

HANDS ON

Vamos começar entendendo o conceito de roteamento no Single SPA. Vou te mostrar como o Single SPA lida com a navegação entre diferentes microfrontends, explicando a importância desse processo para a coesão da aplicação. Além disso, faremos uma configuração básica de roteamento, definindo rotas estáticas no root config. Este é o primeiro passo para garantir que seus microfrontends sejam carregados corretamente conforme o usuário navega pela aplicação.

A seguir, vamos nos aprofundar no roteamento estático. Você aprenderá a criar e configurar rotas estáticas, registrando microfrontends específicos para cada rota definida. Isso é crucial para garantir que cada parte da sua aplicação esteja disponível e funcione perfeitamente em diferentes seções do site.

Por fim, vamos elevar o nível explorando o roteamento dinâmico no Single SPA. Vamos te explicar as vantagens do roteamento dinâmico, mostrando como ele permite uma maior flexibilidade na forma como suas rotas são gerenciadas. Vamos configurar rotas dinâmicas na prática, onde você verá exemplos de como essas rotas podem ser utilizadas para criar experiências de navegação mais ricas e adaptáveis. Assista todas as videoaulas, e assim você dominará o roteamento dinâmico e levará suas habilidades com Single SPA a um novo patamar.

SAIBA MAIS

Autenticação e autorização

Quando pensamos em criar aplicações web seguras, a autenticação e autorização se tornam peças fundamentais do quebra-cabeça. E quando falamos de microfrontends no contexto do Single SPA, essas questões ganham uma complexidade adicional, especialmente ao lidarmos com rotas protegidas.

Primeiro, vamos entender o que cada termo significa. Autenticação é o processo de verificar a identidade de um usuário. Quando um usuário faz login, a aplicação precisa garantir que ele é quem diz ser. Autorização, por outro lado, refere-se a verificar se o usuário autenticado tem permissão para acessar certos recursos ou executar certas ações. Em outras palavras, autenticação responde à pergunta “Quem é você?”, enquanto autorização responde “O que você pode fazer?”.

No contexto de rotas protegidas, esses conceitos se aplicam diretamente. Imagine que você tem uma área administrativa em sua aplicação que apenas usuários com privilégios específicos podem acessar. Não queremos que qualquer pessoa que saiba a URL dessa área possa simplesmente digitá-la no navegador e ter acesso, certo? É aí que as rotas protegidas entram em cena.

Rotas protegidas são aquelas que requerem que o usuário esteja autenticado e, muitas vezes, autorizado a acessar um determinado recurso antes de permitir o carregamento do conteúdo associado a uma rota. No Single SPA, isso significa que, ao tentar acessar uma rota específica, a aplicação verificará se o usuário está autenticado. Se não estiver, ele pode ser redirecionado para uma página de login. Se estiver autenticado, mas não tiver as permissões necessárias, pode ser redirecionado para uma página de acesso negado ou uma página padrão.

A implementação disso no Single SPA pode ser bastante direta, mas há algumas nuances importantes a considerar. Por exemplo, ao proteger uma rota, você precisa garantir que todos os microfrontends que compõem essa rota estejam cientes das regras de autenticação e autorização. Isso pode ser feito por meio de eventos personalizados que disparam verificações de autenticação antes que uma rota seja acessada.

Outro ponto crucial é a gestão do estado de autenticação. Em uma aplicação tradicional, isso pode ser feito por meio de cookies ou tokens armazenados no localStorage, mas em uma arquitetura de microfrontends, é importante garantir que o estado de autenticação seja compartilhado de maneira segura e eficiente entre os diferentes microfrontends. Isso pode ser alcançado por meio de uma store global ou de um serviço de autenticação centralizado que todos os microfrontends possam acessar.

Mas como tudo no desenvolvimento de software, há um equilíbrio a ser encontrado. Enquanto é essencial proteger rotas sensíveis, é igualmente importante garantir que o processo de autenticação e autorização seja rápido e não comprometa a experiência do usuário. Em microfrontends, onde diferentes partes da aplicação podem ser carregadas de forma independente, manter uma navegação fluida enquanto gerencia a segurança pode ser um desafio, mas é totalmente factível com a abordagem certa.

SEO em microfrontends

Quando falamos sobre SEO (Search Engine Optimization) em aplicações web, o objetivo é garantir que nossa aplicação seja facilmente encontrada e bem ranqueada pelos motores de busca, como o Google. No entanto, quando essa aplicação é construída usando uma arquitetura de microfrontends, o cenário se torna um pouco mais complexo, mas não impossível de ser bem gerenciado.

Vamos começar entendendo porquê o SEO pode ser um desafio em microfrontends. Diferente de uma aplicação monolítica tradicional, onde o conteúdo é carregado de uma vez e de forma linear, uma aplicação baseada em microfrontends divide o conteúdo em várias partes, cada uma gerenciada por um módulo ou microfrontend separado. Isso pode dificultar o trabalho dos motores de busca, que precisam rastrear e indexar o conteúdo da página.

Um dos principais problemas é que, em muitos casos, as aplicações de microfrontends são renderizadas no lado do cliente (client-side rendering), o que significa que o conteúdo da página é montado e exibido pelo navegador após o carregamento inicial. Os motores de busca, por sua vez, têm dificuldade em indexar páginas que dependem completamente do JavaScript para carregar o conteúdo, já

que eles podem não esperar que todo o JavaScript seja executado antes de tentar indexar a página.

A renderização no lado do servidor (server-side rendering) pode ser uma solução eficaz para esse problema. Com o SSR, o conteúdo da página é pré-renderizado no servidor antes de ser enviado ao navegador. Isso significa que, quando um motor de busca visita a página, ele vê o conteúdo completo e pronto para ser indexado, o que pode melhorar significativamente o SEO.

Outro aspecto a ser considerado é a coerência do conteúdo. Em uma arquitetura de microfrontends, onde diferentes partes da aplicação podem ser desenvolvidas por equipes diferentes, é importante garantir que todos os microfrontends sigam práticas consistentes de SEO. Isso inclui o uso adequado de tags HTML, como <title>, <meta> e cabeçalhos <h1>, <h2> etc., bem como a criação de URLs amigáveis para cada rota.

Uma abordagem que pode ajudar é a utilização de um design system centralizado que define padrões não apenas para a interface do usuário, mas também para as práticas de SEO. Isso garante que, mesmo que diferentes equipes estejam trabalhando em diferentes partes da aplicação, elas sigam as mesmas diretrizes para otimização de motores de busca.

Por fim, é importante pensar em como compartilhar metadados entre os diferentes microfrontends. Em uma página de microfrontends, cada parte pode precisar contribuir com suas próprias informações de SEO, como descrições, palavras-chave ou mesmo imagens de pré-visualização para redes sociais. Gerenciar isso de forma centralizada ou garantir que os microfrontends possam comunicar essas informações de forma coordenada é crucial para manter uma boa performance de SEO.

Sincronização de estado entre microfrontends

Quando trabalhamos com microfrontends, um dos grandes desafios é garantir que diferentes partes da aplicação possam compartilhar informações de maneira eficaz. Isso nos leva a um conceito crucial: a sincronização de estado. Em um monólito tradicional, é relativamente simples gerenciar o estado da aplicação porque tudo está centralizado em um único lugar. No entanto, quando dividimos a aplicação em microfrontends independentes, a situação se torna mais complexa. Cada

microfrontend pode ter seu próprio estado, mas em muitos casos, eles precisam trocar informações para funcionar corretamente em conjunto.

Imagine, por exemplo, uma aplicação de e-commerce composta por vários microfrontends: um para o catálogo de produtos, outro para o carrinho de compras, e um terceiro para o checkout. Se um usuário adiciona um produto ao carrinho no microfrontend do catálogo, essa informação precisa ser passada para o microfrontend do carrinho de compras e, eventualmente, para o checkout. Essa troca de informações precisa ser feita de maneira rápida e eficiente, garantindo que todos os microfrontends envolvidos estejam sempre na mesma página.

Existem várias maneiras de lidar com essa sincronização de estado. Uma abordagem comum é utilizar um estado global compartilhado, onde todos os microfrontends podem acessar e modificar o estado conforme necessário. Ferramentas como Redux, ou até mesmo uma implementação personalizada utilizando o Context API do React, podem servir como o ponto central onde o estado é armazenado e gerenciado. No entanto, é importante lembrar que essa abordagem, se não for bem gerenciada, pode reintroduzir o acoplamento que estamos tentando evitar ao adotar microfrontends. Um estado global mal projetado pode fazer com que mudanças em um microfrontend impactem inesperadamente outros, dificultando a manutenção e a escalabilidade.

Outra abordagem interessante é a comunicação baseada em eventos. Ao invés de compartilhar diretamente o estado entre os microfrontends, cada microfrontend pode emitir eventos quando algo relevante acontece (como a adição de um item ao carrinho). Outros microfrontends podem “ouvir” esses eventos e atualizar seu próprio estado conforme necessário. Esse modelo de comunicação é mais flexível e mantém os microfrontends mais desacoplados, permitindo que cada um deles seja desenvolvido e atualizado de maneira mais independente.

Também podemos considerar o uso de uma store distribuída, onde o estado é sincronizado entre os microfrontends de forma mais granular e descentralizada. Aqui, em vez de termos um único ponto de verdade, o estado é distribuído entre diferentes partes da aplicação, com mecanismos em vigor para garantir que todos estejam em sincronia. Isso pode ser feito, por exemplo, por meio de APIs de sincronização em tempo real, como aquelas oferecidas por Firebase ou WebSockets.

Manter a sincronização de estado entre microfrontends requer uma arquitetura bem planejada. É preciso garantir que o sistema de sincronização seja resiliente e que tenha maneiras de lidar com possíveis conflitos de estado ou falhas de comunicação. Também é importante documentar como o estado é gerenciado e sincronizado, para que todas as equipes envolvidas tenham um entendimento claro de como as coisas funcionam.



O QUE VOCÊ VIU NESTA AULA?

Nesta aula, nos aprofundamos no roteamento no Single SPA e como ele é essencial para garantir que uma aplicação com múltiplos microfrontends funcione de maneira coesa. Falamos sobre como o Single SPA lida com a navegação, permitindo que diferentes partes da aplicação sejam carregadas e exibidas conforme o usuário interage com as rotas.

Em nossa videoaula, exploramos juntos como configurar o roteamento de forma que cada microfrontend responda corretamente às diferentes URLs que o usuário pode acessar. Isso é fundamental para que cada módulo da aplicação se comporte como esperado, mantendo a experiência de navegação suave e sem interrupções.

Discutimos também a importância das rotas estáticas, que permitem mapear exatamente onde cada microfrontend deve ser carregado. Com essa configuração, você garante que os componentes certos apareçam nos lugares certos, oferecendo ao usuário uma navegação previsível e consistente.

Depois, passamos para o roteamento dinâmico, que adiciona uma camada extra de flexibilidade à aplicação. Falamos sobre como as rotas dinâmicas permitem que a aplicação reaja de maneira inteligente a diferentes cenários e demandas, adaptando o conteúdo exibido de acordo com o contexto, e criando uma experiência de usuário mais rica e interativa.

A cada passo, vimos como configurar essas rotas no Single SPA, entendendo tanto as possibilidades quanto os cuidados necessários para manter a aplicação organizada e eficiente. Agora, você tem as ferramentas e o conhecimento para implementar e otimizar o roteamento em suas aplicações de microfrontends, aproveitando ao máximo a flexibilidade e o poder que essa arquitetura oferece.

REFERÊNCIAS

SINGLE-SPA **Documentation. Devtools.** 2024. Disponível em: <https://single-spa.js.org/docs/devtools/>. Acesso em: 7 ago. 2024.

SINGLE-SPA Documentation. **Ecosystem React.** 2024. Disponível em: <https://single-spa.js.org/docs/ecosystem-react/>. Acesso em: 7 ago. 2024.

SINGLE-SPA Documentation. **Layout Overview.** 2024. Disponível em: <https://single-spa.js.org/docs/layout-overview/>. Acesso em: 14 ago. 2024.

EXEMPLO

PALAVRAS-CHAVE

Roteamento em Microfrontends. Single SPA. Navegação Dinâmica.

EMENDAS

POS TECH